



FOR JAVA DEVELOPERS MOVING TO NESTJS

It Started Broken.

Spring Boot to NestJS, episode 5: configuration and environments

A missing setting starts cleanly here, as undefined. Validate the environment at startup.

In Spring, a required `@Value` refuses to start when its property is missing, on your machine, with the key named. In NestJS the same omission starts cleanly and hands you undefined.

Neither is a bug. The NestJS documentation calls it standard practice to throw at startup when required variables are missing, and shows a schema library to do it.

Versions on this sheet were current on 29 September 2026: Node 22, NestJS 12, Zod 4, Spring Boot 4.1.1.

In scope

- What a missing setting does in each stack
- Why every environment value is a string, and where that bites
- `ConfigService.get` against `@Value`, and relaxed binding
- Validating the environment at startup with Zod
- The boolean trap inside the obvious fix

Not in scope

- Spring profiles (`NODE_ENV` is a convention your code interprets, not a framework profile system)
- Request body validation: that is episode 7

Spring does not fail fast on everything: unvalidated `@ConfigurationProperties` bind null and start, measured.

01 The measurement

NESTJS, NO SCHEMA	starts; first payment: Failed to parse URL from undefined/charges
SPRING, REQUIRED @VALUE	refuses: Could not resolve placeholder 'payment.gateway.url'
SPRING, @CONFIGURATIONPROPERTIES	starts with url = null until @Validated + @NotNull
FEATURE_ENABLED=false	arrives as "false", and the feature is ON
PORT + 1	"30001", not 3001
Z.COERCE.BOOLEAN("false")	true
Z.STRINGBOOL("false")	false
NESTJS, ZOD SCHEMA	refuses: PAYMENT_GATEWAY_URL: expected string, received undefined

The same missing URL, three outcomes.

02 The fix: validate at startup

Pass a Standard Schema to `ConfigModule.forRoot()`. The URL must be a URL, the port is coerced to a number, and the flag uses `z.stringbool()`, because `z.coerce.boolean()` follows JavaScript's `Boolean()` and turns "false" into true.

```
config.schema.ts

import { z } from 'zod';

const schema = z.object({
  PAYMENT_GATEWAY_URL: z.url(),
  PORT: z.coerce.number().int(),
  FEATURE_ENABLED: z.stringbool(),
});

@Module({
  imports: [ConfigModule.forRoot({ validationSchema: schema })],
})
class AppModule {}
```

"It's standard practice to throw an exception during application startup if required environment variables haven't been provided or don't meet certain validation rules."

NestJS documentation, Configuration, checked 2026-09-29.

"Environment variables always arrive as strings"

NestJS documentation, Configuration, checked 2026-09-29.

03 What Spring was doing for you

A required placeholder is resolved while the context is built. `@ConfigurationProperties` binds structured values, and an environment variable written `PAYMENT_GATEWAY_URL` binds to `payment.gateway.url` (relaxed binding). Without `@Validated` it binds null and starts.

```
GatewayPropertiesTest.java

@ConfigurationProperties("payment.gateway")
record GatewayProperties(String url) {}

@Validated
@ConfigurationProperties("payment.gateway")
record ValidatedGatewayProperties(@NotNull String url) {}
```

Unvalidated: started, url = null. Validated: refused to start, "must not be null".

Run on a real machine, 2026-09-29, by `scripts/verify-config.sh` in the course repository.



04 Two traps while wiring it

`ConfigModule.forRoot` reads the environment when the module is declared, not when the application starts: set variables first. `NestFactory` exits on a startup error; with the logger off it exits with no message.

Before you ship NestJS configuration

1. Validate the environment in `ConfigModule.forRoot`
2. Coerce numbers in the schema, never with `+` at the call site
3. Use `z.stringbool()` for flags, not `z.coerce.boolean()`
4. Set variables before the module is declared
5. If the app exits silently, turn the logger on
6. In Spring, add `@Validated` to `@ConfigurationProperties`



Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- Every measurement, one command: scripts/verify-config.sh**
<https://github.com/code-with-sam-dev/spring-to-node>
- NestJS: Configuration**
<https://docs.nestjs.com/techniques/configuration>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-29 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.